

A TOOLBOX TALK

Useful Python tools for building a quantum harmonic oscillator

... to turn an image upside down, and back up again

uv · ruff · ty · pre-commit · array-api-compat

Two talks in a one

A small, real numerical project — used as an excuse to show the tooling that makes building it easier.

01 The physics

- A 2D quantum harmonic oscillator on a periodic grid
- One Hamiltonian, one exponential, and four increasingly good ways to compute it.
- The horse turns upside down and back up again

02 The Python

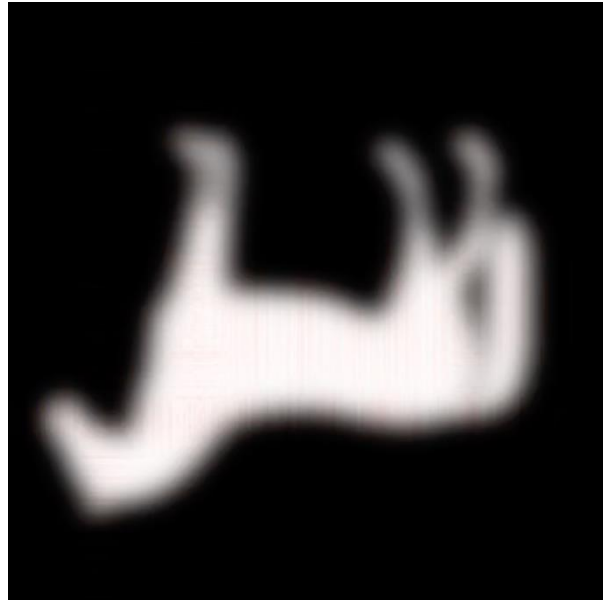
- uv — project, interpreter, dependencies, lockfile
- ruff and ty — lint, format and type-check in Rust time
- pre-commit — checks and fixes run automatically
- array-api-compat — one algorithm, CPU or GPU

Our goal: turn Schrödinger's horse upside down — and back again

At $t = \pi$ the horse is inverted; at $t = 2\pi$ it is exactly back where it started.



$t = 0$



$t = \pi$



$t = 2\pi$

PART ONE

The physics

One Hamiltonian, one exponential, and three increasingly good ways to compute it.

The equation we are solving

Schrodinger Equation describing a state in an infinite well.

$$\frac{\partial \Psi}{\partial t} = i\hat{H}\Psi \quad \hat{H} = T + V$$

$$\Psi(x, y, t) \in \mathbb{C}^{N \times N}$$

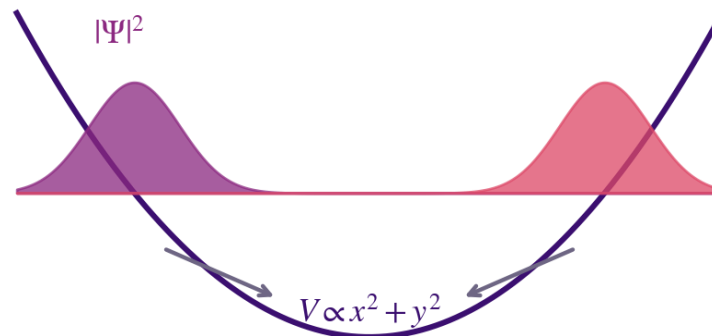
The picture. One complex number per grid point
— modulus is brightness, phase is where it is
going.

$$V(x, y) = -\frac{1}{2}(x^2 + y^2)$$

The infinite well. Depends only on position, so it
is diagonal in position space.

$$T = \frac{1}{2}\nabla^2, \quad \mathcal{F}[T] = -\frac{1}{2}(k_x^2 + k_y^2)$$

Spreading. Depends only on momentum, so it is
diagonal in Fourier space.



The solution is a single exponential

H does not change with time, so the entire evolution is one operator exponential. Every method that follows computes it.

$$\Psi(t) = \exp(i\hat{H}t)\Psi_0$$

Differentiate it and you get $i\hat{H}$ back. That is the verifies the solution.

$$\frac{d}{dt} \exp(i\hat{H}t)\Psi_0 = i\hat{H} \exp(i\hat{H}t) \Psi_0$$

So why is this hard?

N=16,384

unknowns on a 128×128 grid

$N^2 \approx 2.7 \times 10^8$

entries in the matrix $\hat{H}$

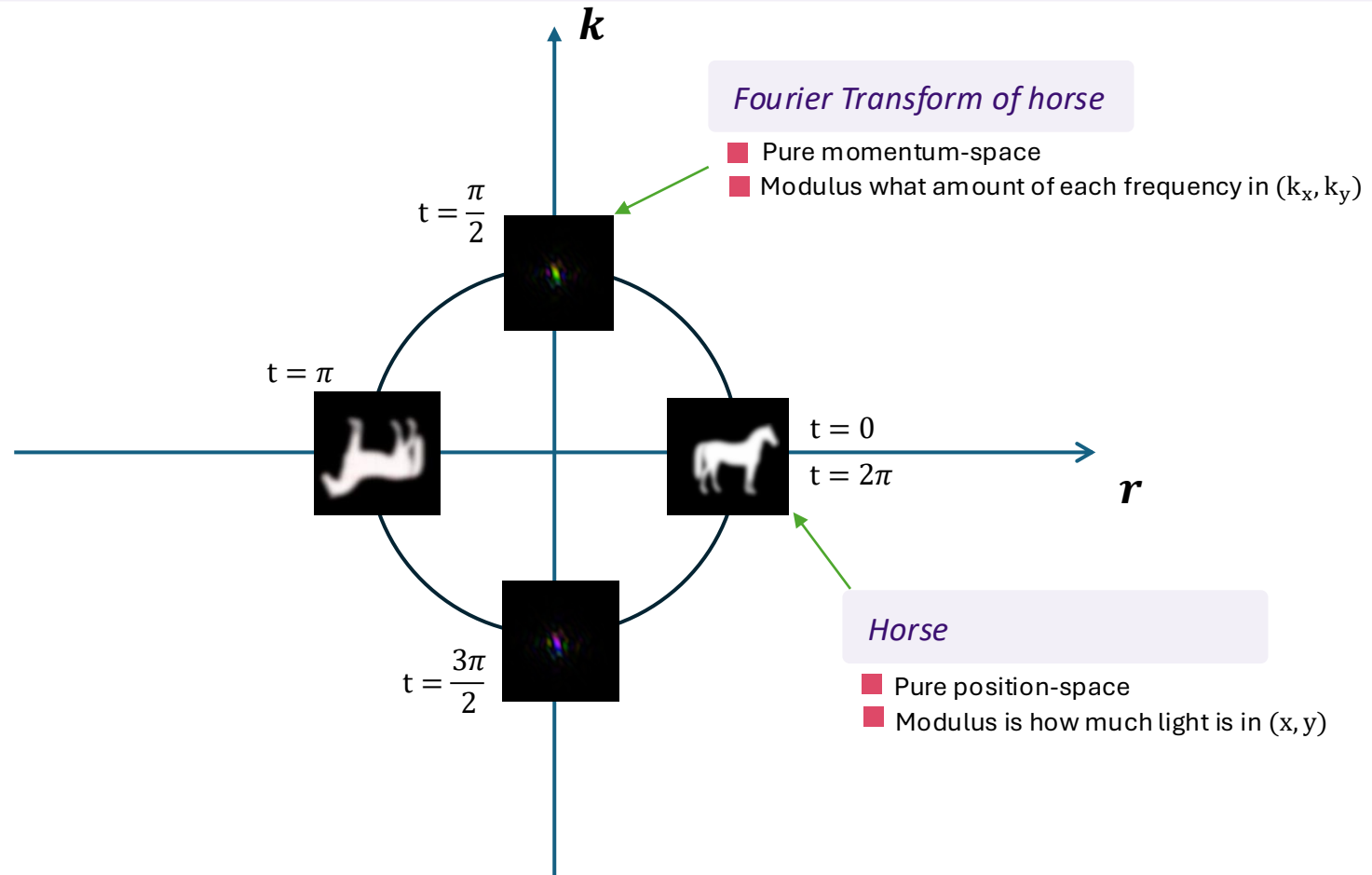
$\mathcal{O}(N^3) \approx 4.4 \times 10^{22}$

operations to compute the dense matrix
exponentiation

So we never build $\hat{H}$ or compute it's exponent directly. Every method below only ever needs $\hat{H}$ applied to a vector.

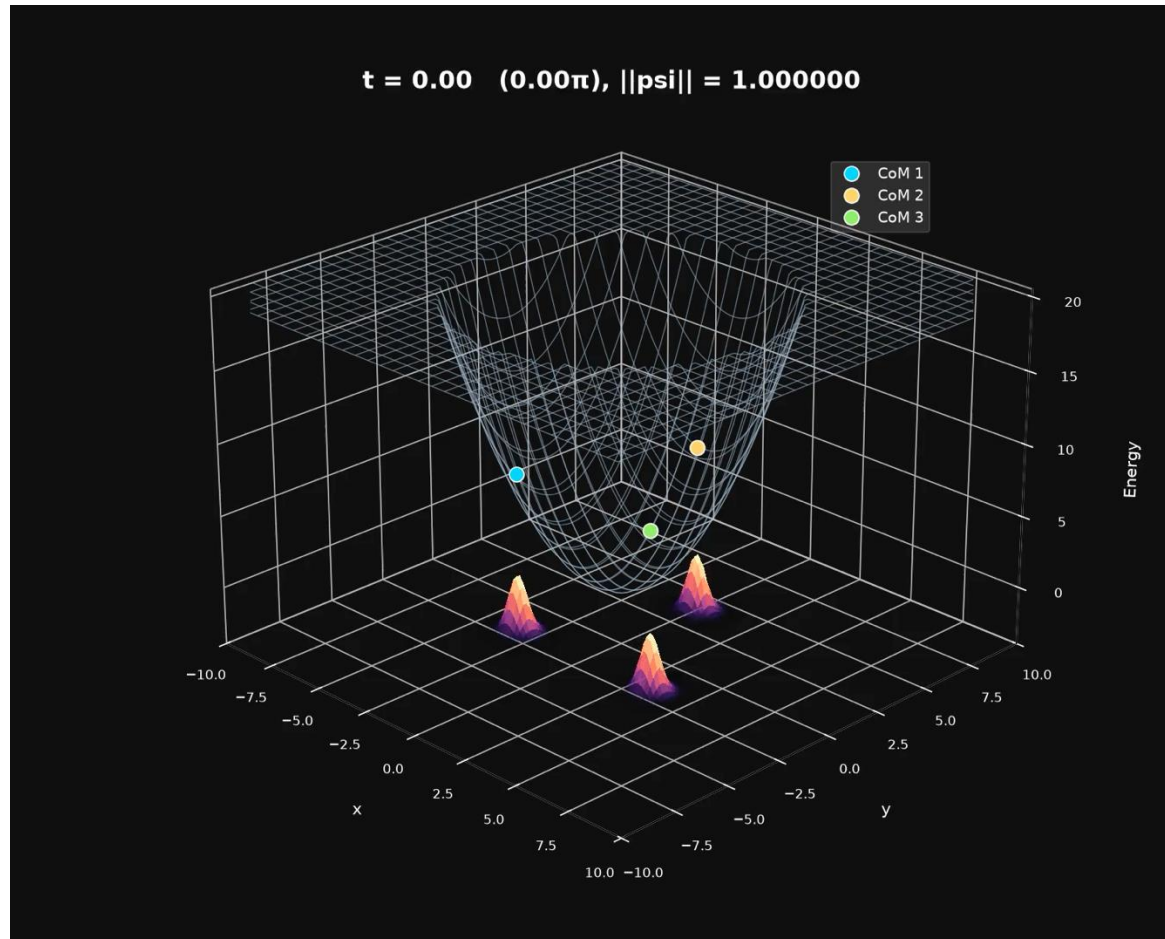
Solution to equations: A rotation in phase space

The oscillator rotates position into momentum. A quarter turn is the Fourier transform; a half turn is the image, inverted.



Solution to equations: A rotation in phase space

The oscillator rotates position into momentum. A quarter turn is the Fourier transform; a half turn is the image, inverted.



Method 1: Forward Euler

Truncate Taylor series after two term. The norm grows at every single step.

$$\exp(i\hat{H}\Delta t) = 1 + i\hat{H}\Delta t + \frac{1}{2}(i\hat{H}\Delta t)^2 + \dots$$

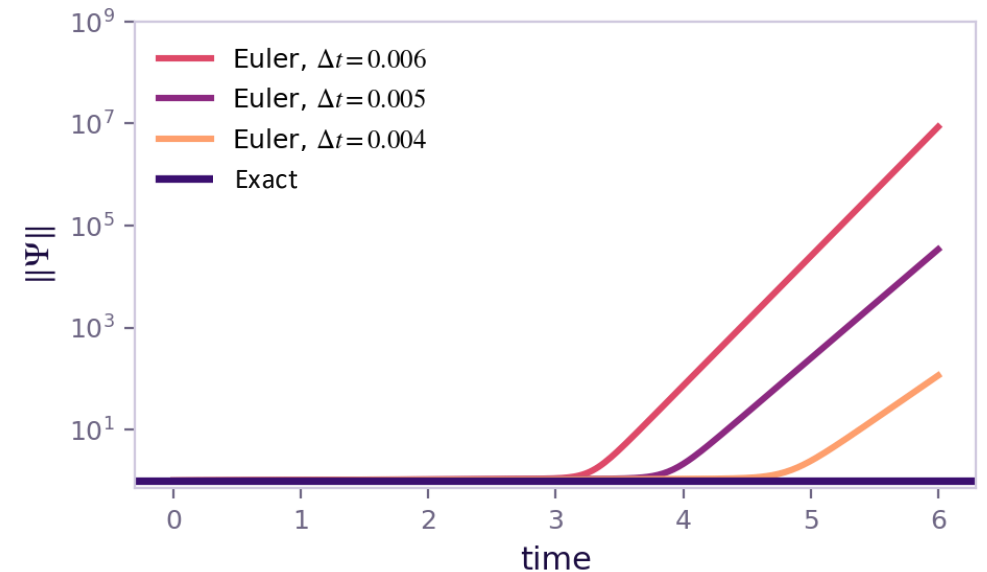
keep two terms

$$\Psi_{n+1} = (1 + i\hat{H}\Delta t)\Psi_n$$

For any eigenvalue E of $\hat{H}$

$$|1 + iE\Delta t| = \sqrt{1 + E^2\Delta t^2} > 1$$

- Bigger than one for every E and every Δt — there is no step size that makes this method conserve the norm
- Halving Δt delays the blow-up. It does not prevent it



Same initial state, same 6 units of time. Only the step size changes.

Method 2: Crank–Nicolson

The same series written as a ratio. Norm-preserving at any Δt — you pay with a linear solve every step.

Approximate the exponential by a ratio (a [1,1] Padé approximant):

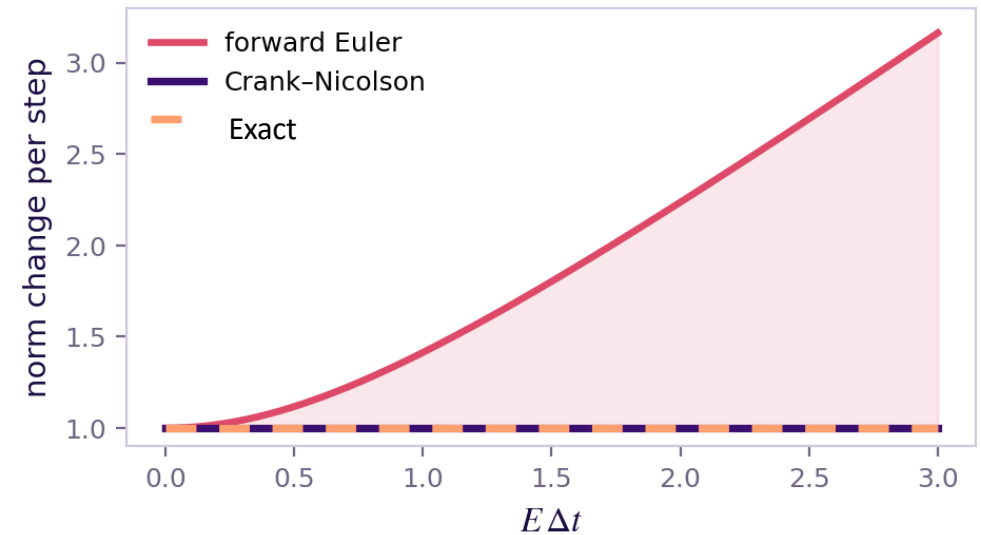
$$\exp(i\hat{H}\Delta t) \approx \frac{1 + \frac{1}{2}i\hat{H}\Delta t}{1 - \frac{1}{2}i\hat{H}\Delta t}$$

which rearranges into one step of the method:

$$\left(1 - \frac{1}{2}i\hat{H}\Delta t\right)\Psi_{n+1} = \left(1 + \frac{1}{2}i\hat{H}\Delta t\right)\Psi_n$$

$$\left| \frac{1 + \frac{1}{2}iE\Delta t}{1 - \frac{1}{2}iE\Delta t} \right| = 1 \text{ for every } E, \Delta t$$

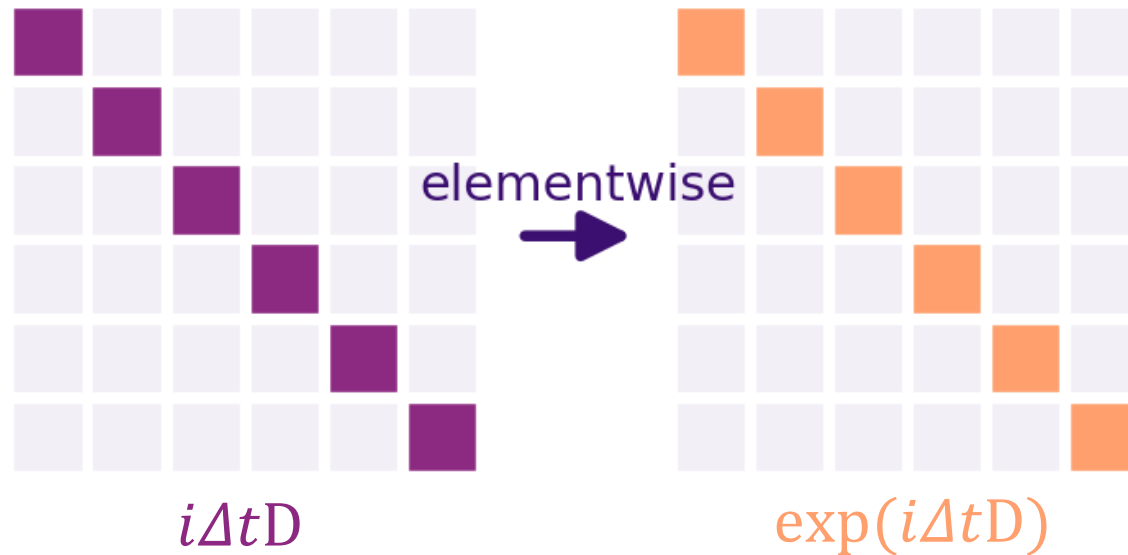
Top and bottom have the same modulus, so the norm is preserved exactly.



- Second-order accurate, stable at any step size
- The cost: the left-hand side is not diagonal, so every step is an iterative solve over 16,384 unknowns

The loophole: the exponential of a diagonal operators is cheap

If an operator is diagonal, its exponential is just the exponential of each entry. No matrix exponential required.



Exponentiate the eigenvalues.

$$\exp(i\Delta t D)_{jj} = \exp(i\Delta t d_j)$$

- An $N^2 \times N^2$ matrix exponential becomes N^2 scalar exponentials
- So the game is to find a basis where the operator is diagonal

And if a change of basis diagonalises it, the exponential changes basis too (verify by substituting into the Taylor series)

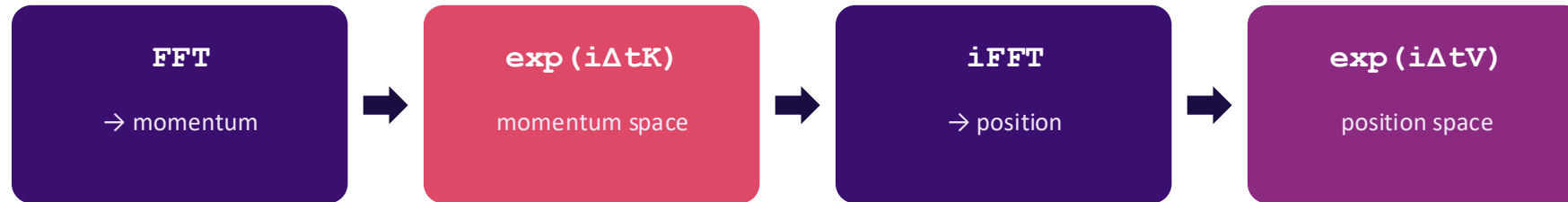
$$T = F^\dagger K F \quad \Rightarrow \quad \exp(i\Delta t T) = F^\dagger \exp(i\Delta t K) F$$

On a periodic grid, F is the FFT — and the kinetic operator T is diagonal in Fourier space.

Method 3: Operator splitting (Lie-Trotter)

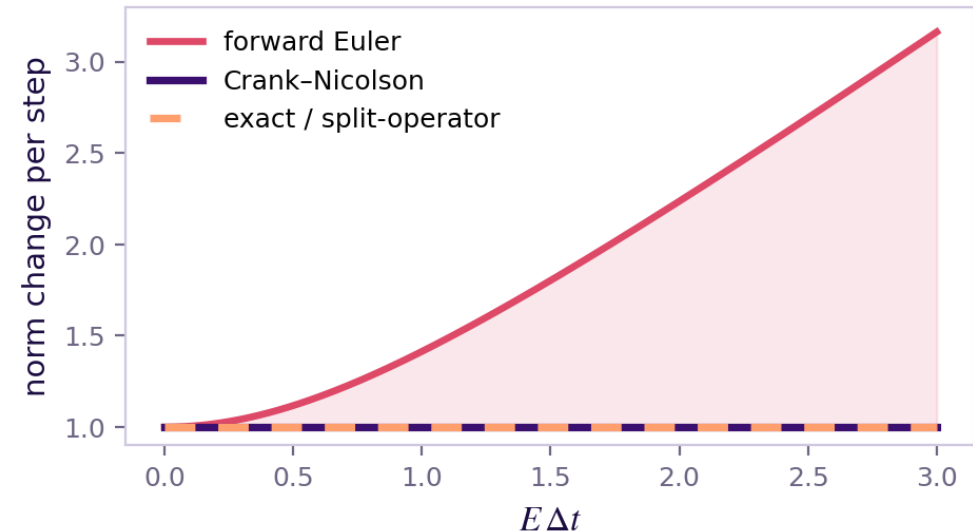
V is diagonal in position. T is diagonal in momentum. FFT between the two and both parts are elementwise multiplies.

$$\exp(i\Delta t(T + V)) \approx \exp(i\Delta t T)\exp(i\Delta t V)$$



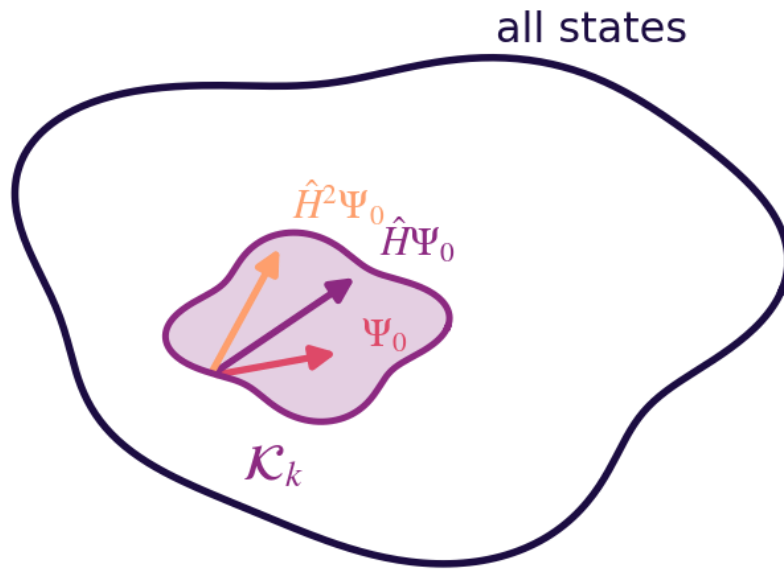
one step — two FFTs, three elementwise multiplies

- Every factor is unitary, so the product is unitary: the norm is conserved exactly, at any Δt
- The error since **T and V do not commute** — $O(\Delta t)$ error globally



Method 4: Arnoldi — time steps at machine precision

Build a small subspace that already contains most of the answer, exponentiate the small matrix exactly, then project back.



Repeatedly applying $\hat{H}$ sweeps out the directions the state actually visits.

1 Build the basis

$$\mathcal{K}_k = \text{span}\{\Psi_0, \hat{H}\Psi_0, \hat{H}^2\Psi_0, \dots, \hat{H}^{k-1}\Psi_0\}$$

k applications of the Hamiltonian — each one is an FFT pair.

2 Project H into it

$$H_k = V_k^\dagger \hat{H} V_k \quad (k \times k)$$

An upper triangular Hessenberg $k \times k$ matrix.

3 Exponentiate exactly, project back

$$\exp(i\Delta t \hat{H})\Psi_0 \approx \|\Psi_0\| V_k \exp(i\Delta t H_k) e_1$$

Small enough that a dense matrix exponential is cheap and accurate.

Method Comparison

Split-operator wins here as it conserves the solution norm and does not require a linear solve.

Method	Cost per step	Stable?	Norm
Forward Euler	2 FFTs/ Matvec product	Small Δt only	Grows
Crank–Nicolson	A linear solve	Any Δt	~Conserved.
Lie-Trotter	2 FFTs	Any Δt	~Conserved.

PART TWO

The Python toolbox

uv · ruff · ty · pre-commit · array-api-compat

The toolchain at a glance

Four of these keep the repo clean and reproducible. The fifth is what lets the same maths run on a laptop CPU and on a GPU.



uv

Project, interpreter, dependencies, lockfile

```
uv sync
```



ruff

Lint and formatter — replaces flake8, isort, black

```
ruff check --fix
```



ty

Type checker

```
ty check
```



pre-commit

Runs all of the above on every commit

```
pre-commit install
```



array-api-compat

One algorithm; NumPy, PyTorch or CuPy underneath

```
xp = array_namespace(psi)
```


uv — the project, the Python, and the lockfile

git clone → uv sync → running. One binary replaces pip, pip-tools, pipx, virtualenv, pyenv and poetry.

TERMINAL

```
$ uv init          # pyproject.toml, .venv
$
$ uv add numpy matplotlib array-api-compat
$ uv add --dev ruff ty pre-commit
$
$ uv sync          # exact env from uv.lock
$ uv ruff check .  # run a tool, install nothing
```

pyproject.toml — written for you

```
[project]
dependencies = ["numpy", "matplotlib", "array-api-compat"]

[dependency-groups]
dev = ["ruff", "ty", "pre-commit"]
```

- Resolves and installs 10–100× faster than pip
- uv.lock is universal: the same resolution on my Mac and on the CI runner
- uv run activates nothing and forgets nothing
- It manages the interpreter too, so “which Python?” stops being a question

ruff — lint and format in one pass

The formatter is not configurable enough to argue about, which is the entire point.

pyproject.toml

```
[tool.ruff]
line-length = 88
target-version = "py313"

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B", "NPY", "SIM"]
```

TERMINAL

```
$ ruff check . --fix      # lint, and fix what it can
$ ruff format .          # formatting
```

- One tool instead of flake8 + isort + pyupgrade + pydocstyle + black
- NPY rules catch legacy NumPy patterns — useful when half the repo is array code
- Fixes are applied, not just reported
- Fast enough to run on save, so the diff never contains formatting noise

ty — type checking without the wait

Types are documentation the machine checks.

TERMINAL

```
$ uv add --dev ty
$ ty check
```

```
error[invalid-argument-type]: argument to
  `split_operator_step` is incorrect:
  expected `Array`, found `ndarray | None`
--> src/toolbox_talk/physics.py:48:22
```

- Rust-based, 10–100× faster than mypy on a cold check, and incremental in the editor
- Beta on the 0.x line — pin the version anywhere it gates CI
- Checks unannotated function bodies by default, unlike mypy: expect a pile of findings on day one
- Gradual guarantee — annotating working code should not invent new errors

pre-commit — make it all automatic

Set up hooks that run automatically each commit. Documentation: <https://pre-commit.com>

.pre-commit-config.yaml

```
repos:
- repo: https://github.com/astral-sh/uv-pre-commit
  rev: 0.12.3
  hooks:
    - id: uv-lock

- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.16.2
  hooks:
    - id: ruff-check
      args: [--fix]
    - id: ruff-format

- repo: local
  hooks:
    - id: ty
      name: ty check
      entry: uv run ty check
      language: system
      types: [python]
```

- pre-commit install once — then every commit is linted, formatted and type-checked
- pre-commit run --all-files for the first, big cleanup
- The same config runs in CI, so local and CI cannot disagree
- autoupdate bumps the pinned tool versions for you

array-api-compat — one algorithm, every backend

The maths never changes. Only the array you hand it does.

```
Lie_trotter_step()
```

the maths written once

`get_namespace(psi)`



NumPy

CPU, float64



PyTorch

CPU / MPS / CUDA



CuPy

CUDA, ROCm



Dask, JAX, ndonnx, sparse

and the rest

■ A common surface

`xp.exp`, `xp.abs`, `xp.fft.fft2` — the standard fixes the names and the semantics, so the algorithm reads the same everywhere.

■ Seven backends

NumPy, CuPy, PyTorch, Dask, JAX, ndonnx and sparse, all wrapped against the 2024.12 version of the spec.

■ The data decides

The namespace comes from the array you were handed — so there is no global backend switch to get wrong.

Example — operator splitting algorithm

Two characters change: np becomes xp — and xp comes from the data.

BEFORE — NUMPY ONLY

```
import numpy as np

def lie_trotter_step(psi, V, K, dt):
    # Kinetic phase shift in momentum space
    # momentum space -> Kinetic -> position space
    psi = np.fft.fft2(psi)
    psi = np.exp(1j * K * dt)
    psi = np.fft.ifft2(psi)

    # Potential wave phase shift in real space
    return psi * np.exp(1j * dt * V)
```

AFTER — ANY BACKEND

```
from array_api_compat import array_namespace

def lie_trotter_step(psi, V, K, dt):
    # Get the appropriate array namespace
    xp = get_namespace(psi)

    # Kinetic phase shift in momentum space
    # momentum space -> Kinetic -> position space
    psi = xp.fft.fft2(psi)
    psi = xp.exp(1j * K * dt)
    psi = xp.fft.ifft2(psi)

    # Potential wave phase shift in real space
    return psi * xp.exp(1j * dt * V)
```

- array-api-compat ships fft wrappers for NumPy, CuPy and PyTorch
- No if backend == ... branches in the physics

TAKEAWAYS

What to take from this

- `uv init` A creates reproducible project (quickly).
- `ruff + ty in pre-commit` The repo stays clean without little effort.
- `xp = array_namespace(psi)` Your maths outlives your hardware.

GitHub: <https://github.com/bryceshirley/quantum-oscillator>

Questions?